Normal Forms and Complete Relations for Circuits in Agda

Xiaoning Bian Yuan Feng

POPL 2027

Tsinghua University

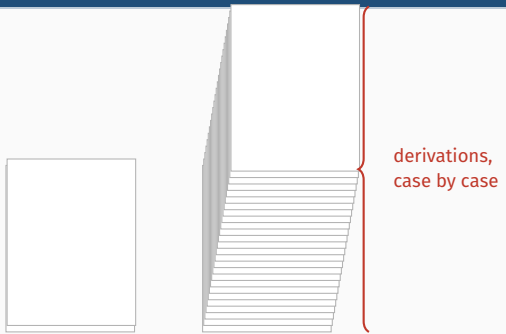
Completeness proofs are gigantic computations

A rule set for circuits is complete when it derives every true equation:

$$[\![w]\!] = [\![v]\!] \implies w \approx v.$$

Soundness is a finite check. Completeness is not:

- define a normal form;
- show every rule, meeting every shape of normal form, reduces again — mechanical, unforgiving, enormous;
- show distinct normal forms mean distinct things.



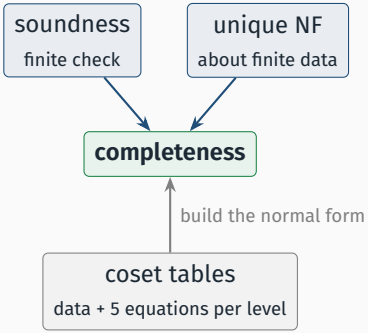
Real-Clifford+CH (Clément). Qutrit Clifford (Li et al.): more than 50 pages of proofs. Long, regular, mechanical: what proof assistants are good at.

This paper

An **Agda library** for presentations, normal forms, and completeness — with circuits as the motivating instance.

- a circuit is a word; a rule set is a datatype; the theory is their inductive congruence;
- a normal form is a function with a section;
- **completeness = soundness + unique normal form**, proved once;
- normal forms grow one wire at a time by verified coset tables, whose obligations often close by `refl`.

Plain, setoid-based Agda on the standard library; `--safe`.



Validated on S_n, $\mathbb{Z}_N \wr S_n$, Pauli groups, qubit and qutrit Clifford+T, $U_3(\mathbb{Z}[\frac{1}{2}, i])$.

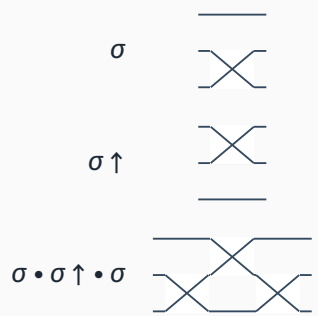
A running example: swap circuits

Circuits are words of wire-indexed generators

A gate set is a family of gates by arity. Here: one gate, the swap.

```
data Gate : ℕ → Set where
  σ-gate : Gate 2
data Gen : ℕ → Set where
  gate₁ : Gate 1 → Gen (1 + n) -- on the bottom wire
  gate₂ : Gate 2 → Gen (2 + n) -- on the bottom two wires
  _↑ : Gen n → Gen (1 + n)     -- one wire higher
```

`Circuit` n = `Word` (`Gen` n): a single generator, the empty circuit ε, or a composition $c \bullet c'$. One constructor of `Gen` serves every width from two up.



wire 0 at the bottom; time flows left to right;

`c ↑ = wmap _↑ c`

Two axioms — and structure for free

```
data _SRel,_===_ : (n : ℕ) → WRel (Gen n) where
  order : (₂₊ n) SRel, σ • σ === ε
  yang-baxter : (₃₊ n) SRel, σ • σ ↑ • σ === σ ↑ • σ • σ ↑
```

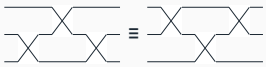
Stated once, at the bottom of the wires, for every width. The structural rules every circuit theory needs come from one generic `Lift-Relation`:

```
data _VRel,_===_ : (n : ℕ) → CRel n where
  srel : n SRel, w === v → n VRel, w === v
  cong↑ : n VRel, w === v → (₁₊ n) VRel, w ↑ === v ↑
  comm₁ : (h : Gate 1) (g : Gen n) →
          (₁₊ n) VRel, [ g ₁ ]ʷ • [ gate₁ h ]ʷ === [ gate₁ h ]ʷ • [ g ₁ ]ʷ
  comm₂ : (h : Gate 2) (g : Gen n) →
          (₂₊ n) VRel, [ g ₁ ₁ ]ʷ • [ gate₂ h ]ʷ === [ gate₂ h ]ʷ • [ g ₁ ₁ ]ʷ
```

order



yang-baxter



shift and disjoint wires commute: the two traces a PROP leaves at fixed width.

Cosets: which wire ends up at the bottom?

S_n sits in S_{n+1} as the circuits that leave the bottom wire alone: the circuits $c \uparrow$. A coset is determined by which wire comes down, so there are $n + 1$ of them, each represented by a staircase.

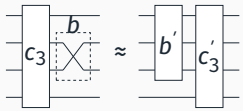
```
data C : ℕ → Set where
  ε : C n
  σ•_ : C n → C (1 + n)
[_]ᶜ : C n → Circuit (1 + n)
[ ε ]ᶜ     = ε
[ σ• c ]ᶜ = σ • ([ c ]ᶜ ↑)
```



One digit per wire: $|S_{n+1}| = (n+1) \cdot n \cdots 2$ — the factorial number system.

The coset table: push one generator past a staircase

```
ract : C (1+ n) → Gen (2+ n) → Circuit (1+ n) × C (1+ n)
ract ε σ-gen = ε , σ• ε                -- grows
ract (σ• ε) σ-gen = ε , ε              -- cancels (σ² = ε)
ract (σ• σ• c) σ-gen = σ , σ• σ• c     -- passes (Yang-Baxter)
ract ε (g 1) = [ g ]ʷ , ε              -- commutes past
ract (σ• c) (g 1) = proj₁ (ract c g) ↑ , σ• (proj₂ (ract c g))
```



staircase, then generator

≈ residual one wire up, then staircase

Each clause is certified by its soundness law:

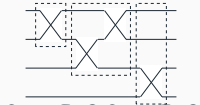
```
ract-sound : ∀ c b → let (b' , c') = ract c b in
             [ c ]ᶜ • [ b ]ʷ ≈ b' ↑ • [ c' ]ᶜ
```



A tower of tables, and the theorem

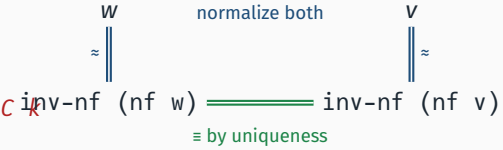
Threading the table through a circuit gives one level of normalization; iterating it, one level per wire, is the coset tower:

```
NF : ℕ → Set
NF 0 = ⊤
NF (suc k) = NF k × C k -- ⊤ × C 0 × C 1 × ⋯ × C k
```



a “staircase of staircases”

`nf` runs the tables; `inv-nf` rebuilds a circuit from its digits. If $[\![w]\!] = [\![v]\!]$:



— a derivation built from `order`, `yang-baxter` and the structural rules only. Even the *endofunction* semantics is complete.

```
symmetric-presentation : ∀ n → (n VRel._===_) IsPresentationOf (Permutation′-group n)
```

Five design decisions

Five bets about where proof effort should go

1 Syntax is unquotiented

everything out of it is a function on words

2 Specifications are stdlib bundles

a normal form *is* an injection, right inverse, or bijection

3 Certificates are finite

a coset table is data + five equations

4 Indices for the unifier

wire counts built from successors only

5 Constructions come in pairs

a syntactic and a semantic theorem

The common thread: the same objects both compute and prove. Normal forms, coset tables and sections are structurally recursive programs that Agda's evaluator runs; their correctness is checked once, generically.

1 Presented monoids are inductive congruences

A word is a syntax tree, *not* a list:

```
data Word (X : Set) : Set where
  [_]ʷ : X → Word X
  ε    : Word X
  _•_  : Word X → Word X → Word X
```

- terms mirror pen-and-paper calculations;
- associativity is a *rule*, discharged by a solver (`by-assoc`: flatten, compare by `refl`);
- groups are monoids whose generators have left inverses (`Grouplike`).

`_===_`: the axioms; `_≈_`: the congruence they generate.

```
data _≈_ : WRel X where
  refl : w ≈ w
  sym  : w ≈ v → v ≈ w
  trans : w ≈ v → v ≈ u → w ≈ u
  cong : w ≈ w' → v ≈ v' → w • v ≈ w' • v'
  assoc      : (w • v) • u ≈ w • (v • u)
  left-unit  : ε • w ≈ w
  right-unit : w • ε ≈ w
  axiom      : w === v → w ≈ v
```

Derivations are data: lifting a whole congruence one wire up, or across a product, is an induction over these eight constructors.

2 Normal forms are standard-library bundles

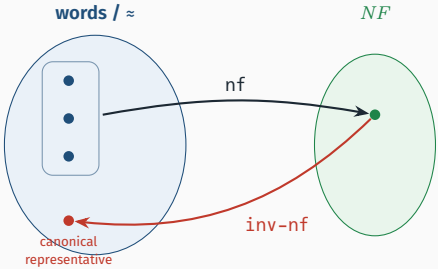
From the word setoid to a carrier NF:

```
NormalFormInjective = Injection
NormalForm          = RightInverse
BijectiveNormalForm = Bijection
```

No new theory, just names for roles. An `Injection` already gives

```
by-equal-nf : nf w ≈ₙ nf v → w ≈ v
≈-dec : Decidable _≈ₙ_ → Decidable _≈_
```

prove equations by computing normal forms; decide the word problem.



Constructively, the section is data

Classically an injective `nf` has a section for free (choose a preimage). In Agda it is an *algorithm* — the extra field of `RightInverse` — and it is what completeness consumes.

Completeness, once and for all

A normal form is unique for a semantics $[\![_]\!]$ (into any setoid) when canonical representatives with equal meanings are equal:

```
record UniqueNormalForm (normalForm : NormalForm) : Set (c ⊔ d) where
  field unique : ∀ {u v : |NF|} → ⟦ inv-nf u ⟧ ≈₂ ⟦ inv-nf v ⟧ → u ≈ₙ v
by-normalization : {normalForm : NormalForm} → UniqueNormalForm normalForm →
  Congruent _≈_ _≈₂_ ⟦_⟧ → Injective _≈_ _≈₂_ ⟦_⟧
```

soundness + unique normal form ⇒ completeness.

Five lines. Every case study proves only `unique` — a statement about normal forms, i.e. finite data, never about arbitrary words.

- `by-completeness`: the converse, given an exact section — how uniqueness travels through products;
- with surjectivity, the records assemble a group isomorphism: `_IsPresentationOf_`.

Verified coset tables

3 Certificates are finite: the Reidemeister–Schreier engine

Γ presents a subgroup *H* of the group *G* presented by Δ. A claimed coset table — cosets *C* with identity *I*, a section [_] : *C* → `Word` *Y*, and a table *h* that pushes a letter past a coset and emits a word of *H* — is verified against five equations; the search that found it stays outside.

```
module Transfer
  (h=⁻¹f-gen : ∀ (x : X) → ([ x ]ʷ , I) ~ ((h ᵗ) I (f x)))                  -- 1
  (h-wd-ax : ∀ (c : C){u t : Word Y} → u ===₂ t → ((h ᵗ) c u) ~ ((h ᵗ) c t))  -- 2
  (f-wd-ax : ∀ {w v} → w ===₁ v → (f ʷ) w ≈₂ (f ʷ) v)                       -- 3
  ([I]≈ε : [ I ] ≈₂ ε)                                                     -- 4
  (h=ract : ∀ c b → let (b' , c') = h c b in [ c ] • [ b ]ʷ ≈₂ (f ʷ) b' • [ c' ]) -- 5
```

1. the table inverts the embedding *f* at *I*;
2. it respects Δ's axioms, coset by coset;
3. *f* respects Γ's axioms;
4. the identity coset is represented by *ε*;
5. coset then letter ≈ emitted word then new coset.

One more digit per level: normal-form transport and towers

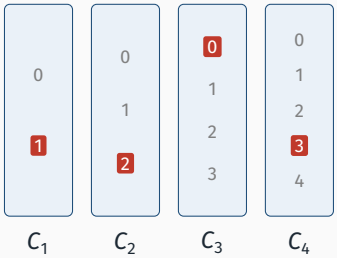
From the five equations, proved once:

```
nf-transp : NormalForm Γ NF
          → NormalForm Δ (NF × C)
```

a normal form for the subgroup gives one for the group, with one more coset digit.
`PackedCosetTable` makes a verified level a first-class value; `CosetTower` stacks them:

$$NF_0 \times C_1 \times C_2 \times \cdots \times C_k$$

— the classical stabilizer-chain normal form, read as a mixed-radix numeral (Schreier–Sims), with syntactically chosen chains: for circuits, *one level per wire*.



S_5: radices $2 \cdot 3 \cdot 4 \cdot 5 = 120$

Where the chain branches — two subgroups over a common base — the gluing is the amalgamated free product, with its alternating normal form.

Obligations close by evaluation

A real level of the single-qubit Clifford+T tower: adjoin X to $\langle\omega\rangle \times \langle S\rangle$, two cosets ε, X. Hypothesis (2), verbatim:

```
h-wd-ax : ∀ c {u t} → u === t → (h ᵗ) c u ~ (h ᵗ) c t
h-wd-ax X-cr {u} {t} order-ω = (by-equal-nf Eq.refl) , Eq.refl
h-wd-ax X-cr {u} {t} order-S = (by-equal-nf Eq.refl) , Eq.refl
h-wd-ax X-cr {u} {t} order-X = (by-equal-nf Eq.refl) , Eq.refl
h-wd-ax X-cr {u} {t} order-SX = (by-equal-nf Eq.refl) , Eq.refl
h-wd-ax X-cr {u} {t} (comm {X-gen}) = (by-equal-nf Eq.refl) , Eq.refl
```

… nine more; from `Examples/Amalgamations/CliffordT1.agda`

Run the table on both sides of the axiom: the cosets coincide, and the residues have the same normal form one level down.

coverage checker
no case is missed

evaluator
closes every case
that is computation

you
the few real derivations,
which read like the paper's

59 obligations: qubit Clifford+T
72 one qutrit level
every one `refl`

4 Indices are chosen for the unifier

`_+_` recurses on its *first* argument:

$2 + n \rightsquigarrow \mathrm{suc}\,(\mathrm{suc}\,n)$ $\qquad$ $n + 2$ is stuck.

So the indices are left-biased: $\mathtt{gate}_2$ lands in `Gen (2 + n)`, and the k-fold shift in `Gen (k + n)`.

Every index equation the coverage checker meets is then solved by injectivity of suc, never by arithmetic — which is why coset tables are written by plain pattern matching.

left-biased ✓

`Gen (2 + n)` $\stackrel{?}{=}$ `Gen (1 + m)`
$\rightsquigarrow$ $m :=$ `1 + n`, by unification

right-biased ×

`Gen (n + 2)` $\stackrel{?}{=}$ `Gen (1 + m)`
$\rightsquigarrow$ $n + 2 \doteq \mathrm{suc}\ m$: stuck,
a Diophantine side condition

5 Constructions come in pairs

One primitive: a three-way join whose mixed part is the parameter.

```
data _◇_◇_ (Γ₁ : WRel A) (Γ₂ : WRel B) (Γ₃ : WRel (A ⊎ B))
           : WRel (A ⊎ B) where
  left  : Γ₁ u v → (Γ₁ ◇ Γ₂ ◇ Γ₃) [ u ]₁ [ v ]₁
  right : Γ₂ u v → (Γ₁ ◇ Γ₂ ◇ Γ₃) [ u ]₂ [ v ]₂
  mid   : Γ₃ u v → (Γ₁ ◇ Γ₂ ◇ Γ₃) u v
```

mixed part	construction
`CommRel`	direct product
`ConjRel`^{w} *conj*	semidirect product
`AmalgRel` f_1 f_2	amalgamated free product
`SugarRel`	definitional extension

Syntactic: normal forms combine — for × and ⋊ the carrier is $NF_1 \times NF_2$; an amalgam has a base normal form and alternating cosets:

```
amalNFC C D = (D ⊎ T) × List (C × D) × (C ⊎ T)
```

Semantic: if the factors present G_1, G_2, the construction presents $G_1 \times G_2$, $G_1 \rtimes G_2$, $G_1 *_{G_0} G_2$ — and group extensions.

Semidirect products, amalgamated free products of monoids and group extensions were not in agda-stdlib; we contribute them (`ForStdlib`, 865 lines).

A catalogue of verified presentations

What the library has verified

relation set	presents	what it exercises
$T^{n+1} = \varepsilon \;/\; T^0 = \varepsilon$	$\mathbb{Z}_{n+1}$ / the *monoid* $(\mathbb{N}, +, 0)$	monoid vs. group presentations
`order`, `yang-baxter` + structure	permutations of Fin n	circuits, a recursive tower
$(\mathbb{Z}_N)^n \rtimes S_n$-circuits	$\mathbb{Z}_N \wr S_n$, every N, n	products with semantics
$(\mathbb{Z}_p \times \mathbb{Z}_p)^n$	Pauli group mod phases, every prime p	pure composition
qubit Clifford+T rules	$\cong \langle T, M\rangle *_M \langle H, M\rangle$	amalgamation, `refl`
qutrit Clifford+T rules	$\cong$ an amalgam over $M = \langle S, X, \zeta, H^2\rangle$	long towers, twisted relations
$U_3(\mathbb{Z}[\frac{1}{2}, i])$, S1–S14	$\cong (\mathbb{Z}_4 \wr S_3) *_M \langle M, K_{01}\rangle$	reuse of the wreath product

Every theorem restated with its full type in one index module, `MainTheorems.agda`.

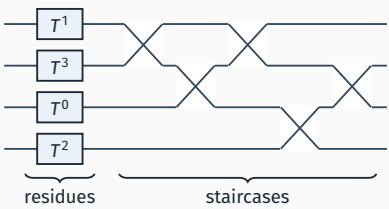
$T^0 = \varepsilon$ presents $\mathbb{Z}$ as a *group* but $\mathbb{N}$ as a *monoid*: words have no formal inverses, and `Grouplike` is exactly what fails.

Whole families at once

Wreath products: signed permutations at $N = 2$, monomial matrices with N-th roots of unity in general — completeness over two naturals:

```
wreath-presentation : ∀ n m →
  (((suc m Cn,_===_) ⊕^ n) ⋄ (n VRel,_===_) ⋄ ConjRel^W conj)
    IsPresentationOf (Wreath.wreath-group n m)
```

normal form of $\mathbb{Z}_N \wr S_n$:



Pauli groups for every prime: three construction theorems composed, no coset enumeration at all:

```
pauli-presentation : ∀ m (prime : Prime (2+ m)) n →
  (Γ-Pauli₁ m prime ⊕^ n) IsPresentationOf (Pauli-group m prime n)
```

normal form of the Pauli group: $X^{a_1}Z^{b_1} \cdots X^{a_n}Z^{b_n}$ — nothing proved about Paulis specifically

Amalgamation rediscovers Matsumoto–Amano

Single-qubit Clifford+T; generators $T, H, S, \omega, X := HSSH$;
$\omega^8 = S^4 = H^2 = \varepsilon$, $T^2 = S$, $(SH)^3 = (TX)^2 = \omega$, ω central.

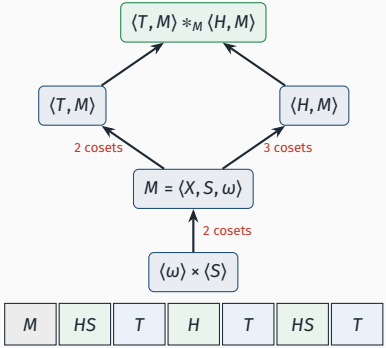
$$\text{Clifford+}T \;\cong\; \langle T, M\rangle *_M \langle H, M\rangle, \qquad M = \langle X, S, \omega\rangle$$

The alternating normal form, read right to left:

$$(\varepsilon \mid T)\,(HT \mid SHT)^*\,(\varepsilon \mid H \mid SH)\,M$$

— Matsumoto–Amano, from the group theory alone.

```
clifford+T-qubit-isomorphism : IsMonoidIsomorphism
  (rawMonoid (•-ε-monoid _===_)) (rawMonoid (•-ε-monoid amalPres)) (f ʷ)
```



a normal form with T-count 3

Same story for qutrit Clifford+T (Glaudell–Ross–Taylor's form) and for $U_3(\mathbb{Z}[\frac{1}{2}, i])$ — where the regular expression is a new canonical form.

Lessons

Setoids, not quotients: the balance sheet

What it cost

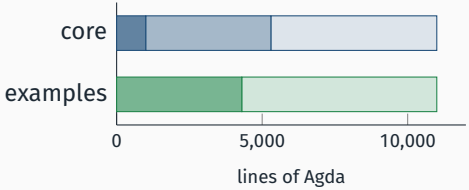
Every function out of a presented monoid carries a congruence proof (bundled by the standard library's `Injection`, `RightInverse`, …).

A univalent quotient would turn ≈ into ≡ — but its eliminator still demands a respect proof for every function out of it.

What it bought

- normal forms, coset tables, sections run: hundreds of obligations close by `refl`;
- maps into the syntax — sections, transversals, amalgamation tails — need concrete representatives;
- plain Agda and the stdlib algebra hierarchy: upstreamable;
- derivations are data: lifting lemmas are inductions.

Size, cost, and what is not formalized



core: presentation core ≈1k, product constructions 4.3k, rest

examples: qutrit Clifford+T and U_3 4.3k, others

≈22 000 lines, 74 files. Whole development from scratch: **1 min 42 s**, 3.5 GB (Agda 2.8.0).

Limitations

- the amalgamation studies stop at syntax: the isomorphism with the matrix groups is taken from the literature — the rings and unitary matrices are not in the standard library;
- continuous parameters (phase gates with real angles) have no finite coset structure: vocabulary yes, engine no;
- infinite-index steps ($\mathbb{Z}_N \wr \mathbb{Z}$) are statable, but need induction instead of finitely many evaluations.

Related work

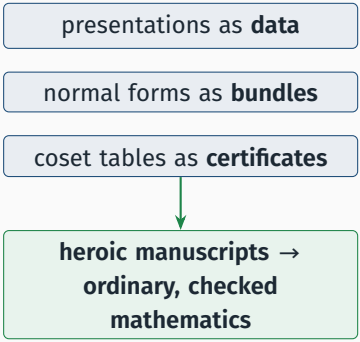
	prover	circuits	presentations	normal forms	completeness
this work	Agda	✓	✓	✓	✓
Bian–Selinger developments	Agda	✓	✓	✓	syntactic
SQIR/VOQC, VyZX	Rocq	✓	–	–	sound only
Π as free rig groupoid	Agda (HoTT)	✓	categorical	✓	classical
$\sqrt{\Pi}$	Agda (partial)	✓	categorical	–	2q Clifford
mathlib group theory	Lean	–	✓	–	–
free groups (AFP)	Isabelle	–	✓	✓	–
IsaFoR completion	Isabelle	–	–	✓	–

vs. abstract presentation theory: no infrastructure to prove that a *given* relation set presents a *given* group — no verified coset enumeration, no transversals.

vs. completion: we never orient rules or prove confluence; a normal form is any verified *function* — avoiding Squier's obstruction.

Takeaways

1. **Completeness = soundness + unique normal form** — one generic lemma; case studies only reason about finite data.
2. **Coset tables are certificates**: finite data plus five equations, checked by coverage and closed by evaluation.
3. **Normal forms grow one wire at a time** — mixed-radix numerals over the chain the circuit layer hands us.
4. **Constructions come in pairs**, against groups we added to the standard library.



Thank you!

Every theorem in the paper is checked by Agda:
`--safe`, `--cubical-compatible`, no postulates,

indexed in `MainTheorems.agda`.